

Lecture 9: Conditioning; Floating-Point Numbers

Math 6610

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Today's Goals

- Separate two sources of error: the problem, and the algorithm
- Define the condition number $\kappa(x)$ and read it as lost digits
- Begin floating point: why real numbers need it

New Topic: How Much Accuracy Is Possible?

Error in a computed answer has two sources:

$$\text{error} = \underbrace{\text{sensitivity of the } \textit{problem}}_{\text{conditioning} - \text{today}} \times \underbrace{\text{quality of the } \textit{algorithm}}_{\text{stability} - \text{later}}.$$

Remark (*Why this comes first*). If the problem is sensitive, no algorithm helps: the data itself does not determine the answer that accurately.

A problem is a map from input to output,

$$F : X \rightarrow Y, \quad x \in X \text{ (input)}, \quad y = F(x) \in Y \text{ (output)}.$$

Example (*Our running case*). Solve $Ay = b$: $x = (A, b)$ with $A \in \mathbb{C}^{n \times n}$ nonsingular and $b \in \mathbb{C}^n$; $Y = \mathbb{C}^n$; $F(A, b) := A^{-1}b$.

The Condition Number

Change the input, watch the output:

$$x \rightarrow x + \delta x, \quad \delta F(x) := F(x + \delta x) - F(x).$$

Definition (condition number)

$\kappa = \kappa(x)$ is the smallest number with

$$\frac{\|\delta F(x)\|}{\|F(x)\|} \leq \kappa(x) \frac{\|\delta x\|}{\|x\|} \quad \text{for all small } \delta x,$$

that is,

$$\kappa(x) := \limsup_{\delta \rightarrow 0} \sup_{\|\delta x\| \leq \delta} \frac{\|\delta F(x)\|}{\|F(x)\|} \frac{\|x\|}{\|\delta x\|}.$$

Relative error in, relative error out. The norms on X and Y are chosen to suit the problem.

Reading κ : How Many Digits Survive

Definition (Well- and ill-conditioned). The problem is *ill-conditioned* at x if $\kappa(x) \gg 1$, and *well-conditioned* otherwise.

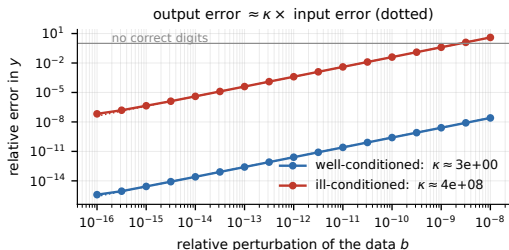
Data known to ≈ 16 digits ($\epsilon_{\text{mach}} \approx 10^{-16}$):

$\kappa(x)$	1	10^2	10^8	10^{14}	10^{16}
digits you can trust	16	14	8	2	0

$$\text{lost digits} \approx \log_{10} \kappa(x).$$

Remark (How to use this). Estimate κ before blaming the solver. If $\kappa = 10^{14}$ and you need 6 digits, change the problem, not the algorithm.

Conditioning, Measured



```
A = [1 1; 1 1+1e-8];  
y = [1; 1]; b = A*y;  
d = 1e-14*norm(b)*[1;-1]/sqrt(2);  
norm(A\b(b+d) - y)/norm(y)  
% ans = 4.0e-06  
cond(A)  
% ans = 4.0e+08
```

- Change b in the last digit (10^{-14}): the answer moves by $4 \cdot 10^{-6}$. Eight digits gone, and \ did nothing wrong.
- κ is a property of the problem, not of the algorithm. In MATLAB, `cond(A)`.

Computing κ for Differentiable F

Remark (The formula). Let $X \subset \mathbb{R}^n$, $Y \subset \mathbb{R}^m$, and let F be differentiable at x . Then

$$\kappa(x) = \|DF(x)\| \frac{\|x\|}{\|F(x)\|}$$

with $\|DF(x)\|$ the matrix norm induced by the chosen vector norms. The recipe: differentiate, take norms, normalize.

Where it comes from

Taylor gives $\delta F(x) = F(x + \delta x) - F(x) = DF(x) \delta x + o(\|\delta x\|)$, so

$$\frac{\|\delta F(x)\|}{\|\delta x\|} \cdot \frac{\|x\|}{\|F(x)\|} \longrightarrow \frac{\|DF(x) \delta x\|}{\|\delta x\|} \cdot \frac{\|x\|}{\|F(x)\|} \stackrel{\sup_{\delta x}}{=} \|DF(x)\| \frac{\|x\|}{\|F(x)\|},$$

the last step being the definition of the induced matrix norm.

Example: $\sqrt{x}$ Is Well-Conditioned

Example. Evaluate $y = F(x) = \sqrt{x}$ for $x > 0$. Then $F'(x) = \frac{1}{2\sqrt{x}}$, so

$$\kappa(x) = |F'(x)| \frac{|x|}{|F(x)|} = \frac{1}{2\sqrt{x}} \cdot \frac{x}{\sqrt{x}} = \frac{1}{2}.$$

Well-conditioned for every $x > 0$: a relative change in x produces half as much relative change in $\sqrt{x}$.

Remark. κ does not depend on x here. That is unusual, and it is why square roots never cause trouble on their own.

Example: $x_1 - x_2$ Can Be Ill-Conditioned

Example. Compute $y = F(x) = x_1 - x_2$ for $x = [x_1 \ x_2]^T \in \mathbb{R}^2$, so $F : \mathbb{R}^2 \rightarrow \mathbb{R}$ and $DF = [1 \ -1]$. In the 2-norm, $\|DF\|_2 = \sqrt{2}$, hence

$$\kappa(x) = \|DF(x)\|_2 \frac{\|x\|_2}{|F(x)|} = \frac{\sqrt{2} \sqrt{x_1^2 + x_2^2}}{|x_1 - x_2|}.$$

Ill-conditioned when $x_1 \approx x_2$ and $x_1, x_2 \neq 0$.

Remark (Why).

$$F(x + \delta x) = \underbrace{x_1 - x_2}_{\approx 0} + \delta x_1 - \delta x_2 \approx \delta x_1 - \delta x_2 :$$

the answer is made of the perturbations. This is *loss of significant digits*.

Loss of Significant Digits

Take $x_1 = 1.000001$, $x_2 = 1.000000$:

$$\kappa(x) = \frac{\sqrt{2} \sqrt{x_1^2 + x_2^2}}{|x_1 - x_2|} \approx 2 \cdot 10^6.$$

Change each input in its 7th digit and the output 10^{-6} changes completely.

Remark. Nothing was computed wrong. The difference carries only what is left after the shared leading digits cancel, and the data does not pin that down.

Remark (*What an algorithm can and cannot do*). Cancellation is a property of the problem. A good algorithm can avoid *creating* a subtraction of nearly equal numbers (Lecture 12); none can repair one the problem itself demands.

New Topic: Representing Real Numbers

Problem: how do we store $x \in \mathbb{R}$ on a computer?

Storage is finite, so $\sqrt{3} = 1.73205\dots$, and even large integers, cannot be stored exactly.

Pick a base $\beta \geq 2$: $\beta = 2$ binary, $\beta = 10$ decimal. For an integer $x \geq 0$,

$$\begin{aligned}x &= b_n \beta^n + b_{n-1} \beta^{n-1} + \dots + b_1 \beta + b_0 \\&=: (b_n b_{n-1} \dots b_1 b_0)_\beta, \quad b_i \text{ integers, } 0 \leq b_i \leq \beta - 1.\end{aligned}$$

Example (*53 in base 10 and base 2*).

$$\begin{aligned}53 &= 5 \cdot 10^1 + 3 \cdot 10^0 = (53)_{10} \\&= 32 + 16 + 4 + 1 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\&= (110101)_2 \quad \Rightarrow \quad 6 \text{ bits.}\end{aligned}$$

Why Floating Point, Not Fixed Point

Example (*How many numbers fit in 52 bits?*). With $x = (b_{51}b_{50} \cdots b_1b_0)_2$ we get $x = 0, 1, 2, \dots, 2^{52} - 1$: about $4.5 \cdot 10^{15}$ values, spaced 1 apart.

That spacing is *absolute*, and scientific computing needs 10^{-300} and 10^{300} in the same program.

Remark (*The floating-point idea*). Fix the number of significant digits and let an exponent place the binary point. The representable numbers then have constant *relative* spacing $\approx 2^{-52}$, which is how error actually propagates through multiplication and division. It is also why conditioning and stability are stated in relative errors.

IEEE Floating Point: the Normalized Form

Base $\beta = 2$. Every $x \in \mathbb{R}$ with $x \neq 0$ can be written as

$$x = \pm(1.b_1b_2\cdots) \times 2^p, \quad b_1, b_2, \cdots \in \{0, 1\}, \quad p \in \mathbb{Z}.$$

Remark. The leading 1 is forced by normalization, so it need not be stored: one free bit of precision. What remains is bookkeeping – how many bits for b_i , how many for p , and what to do with 0, $\pm\infty$, and NaN.

Looking Ahead

- **Next lecture:** IEEE Floating-Point Numbers and Machine Precision